

Atividade Prática 2: Implementação de Algoritmo de Detecção de Comunidades em Redes para uso do conda e do git

1 Objetivo

Com o objetivo principal de exercitar nossas habilidades em Python, manipulação de matrizes, gerenciamento de ambientes virtuais e controle de versão, vamos compreender o funcionamento de um algoritmo de **Detecção de Comunidades** em redes/grafos (baseado no algoritmo **Label Propagation** - Propagação de Rótulos).

Nesta atividade, você deverá implementar o algoritmo utilizando as bibliotecas que você conhece. Além disso, você deverá configurar um ambiente isolado com **Conda** e gerenciar o repositório do seu projeto via **GitHub**.

Atenção: Não usar de funções prontas de detecção de comunidades. O algoritmo de propagação deve ser implementado por você!

2 O Algoritmo de Propagação de Rótulos (Label Propagation)

A detecção de comunidades é uma tarefa fundamental na análise de redes complexas. O algoritmo de Propagação de Rótulos é um método simples, rápido e intuitivo para particionar os vértices de um grafo em grupos coesos.

Funcionamento Passo a Passo

1. **Inicialização:** Cada vértice v no grafo recebe um rótulo único inicial (por exemplo, seu próprio índice $c_v = v$).
2. **Ordem de Atualização:** Em cada iteração, define-se uma ordem aleatória para visitar os vértices do grafo.
3. **Propagação dos Rótulos:** Para cada vértice v , avaliam-se os rótulos de seus vizinhos diretos. O vértice v assume o rótulo mais frequente entre os seus vizinhos. Em caso de empate, um dos rótulos mais frequentes é escolhido aleatoriamente.

4. **Convergência:** O processo é repetido até que nenhum vértice mude de rótulo ou que o número máximo de iterações pré-definido seja atingido.

3 Fórmulas e Definições Matemáticas

Dado um grafo $G = (V, E)$ representado por sua Matriz de Adjacência $A \in \mathbb{R}^{N \times N}$, onde:

$$A_{ij} = \begin{cases} 1, & \text{se existe a aresta } (i, j) \\ 0, & \text{caso contrário} \end{cases}$$

O novo rótulo do vértice i na iteração $t + 1$, denotado por $c_i^{(t+1)}$, é dado pela moda dos rótulos de seus vizinhos no tempo t :

$$c_i^{(t+1)} = \arg \max_l \sum_{j \in N(i)} \delta(c_j^{(t)}, l)$$

Onde:

- $N(i) = \{j \in V \mid A_{ij} = 1\}$ representa o conjunto de vizinhos do vértice i .
- $\delta(a, b)$ é a função Delta de Kronecker ($\delta(a, b) = 1$ se $a = b$, e 0 caso contrário).
- $\arg \max_l$ seleciona o rótulo l que maximiza a soma (em caso de empate, escolhe-se aleatoriamente entre os mais frequentes).

4 Pseudocódigo

```
ALGORITMO LabelPropagation(A, MaxIteracoes):
    //A eh a matriz de adjacencia
    N = NumeroDeVertices(A)
    Rotulos = ArrayNumPy(0 ate N-1) // Cada no com seu proprio
    rotulo

    Iteracao = 0
    MUDOU = VERDADEIRO

    ENQUANTO Iteracao < MaxIteracoes E MUDOU FACA:
        MUDOU = FALSO
        OrdemVertices = AleatorizarLista(0 ate N-1)

        PARA CADA i EM OrdemVertices FACA:
            Vizinhos = ObterVizinhos(A, i) // Vizinhos onde A[i,
            j] == 1

            SE Vizinhos NAO ESTA VAZIO ENTAO:
```

```

        RotulosVizinhos = Rotulos[Vizinhos]
        NovoRotulo = CalcularModaComEmpateAleatorio(
RotulosVizinhos)

        SE NovoRotulo != Rotulos[i] ENTAO:
            Rotulos[i] = NovoRotulo
            MUDOU = VERDADEIRO
        FIM SE
    FIM SE
FIM PARA
    Iteracao = Iteracao + 1
FIM ENQUANTO

    RETORNAR Rotulos
FIM ALGORITMO

```

Listing 1: Pseudocódigo do Algoritmo Label Propagation

5 Instruções de Implementação e Requisitos

Regras de Ambiente e Versionamento

1. **Uso do Conda:** Você deve criar e gerenciar um ambiente virtual específico para esta atividade utilizando o `conda`. Um arquivo `environment.yml` deve ser incluído na raiz do repositório contendo as dependências exigidas (`numpy`, `networkx`, `matplotlib`).
2. **Uso do GitHub:** A entrega deverá ser realizada via repositório público no GitHub. Espera-se um histórico claro de *commits* demonstrando a evolução do desenvolvimento.
3. **Bibliotecas Permitidas:** É obrigatório o uso de `numpy` (para operações matriciais/vetorização) e `networkx` (para carregar/gerar grafos e exportar vizinhanças ou gráficos).
4. **IA Generativa:** Proibido o uso de ferramentas de IA generativa para escrita direta do código. O foco é fixar a lógica de propagação e o uso correto do NumPy/NetworkX.

6 Arquivos de Teste (Datasets e Redes)

Os arquivos de entrada devem ser salvos em formato CSV na pasta `data/` do repositório.

Arquivo 1: rede1_duas_comunidades.csv

Grafo simples com duas comunidades fracamente conectadas entre si (Ideal para *debug*).

```
origem,destino
0,1
0,2
1,2
2,3
3,4
3,5
4,5
```

Arquivo 2: rede2.csv

Rede clássica adaptada com mais conexões.

```
origem,destino
0,1
0,2
0,3
1,2
1,3
2,3
3,4
4,5
4,6
5,6
0,4
```

—

7 Entregáveis

O aluno deverá entregar no Sigaa uma pasta compactada com o código e o link do repositório GitHub contendo:

1. Código-fonte Python devidamente comentado.
2. Arquivo `environment.yml` exportado via Conda.
3. Arquivo `README.md` contendo:
 - Instruções para clonar o repositório e criar o ambiente conda.
 - Relatório sucinto dos testes executados em cada dataset, com imagens/plots das comunidades geradas (se possível).
 - Principais dificuldades encontradas durante a implementação.